

Aula 3 (Módulo 06) — Projeto: Parser de Log SSH

Nesta aula construímos, do zero, um parser de log SSH de verdade — juntando tudo que vimos nas Aulas 1 e 2: variáveis, condições, loops, funções, arquivos, regex e JSON. O código completo já está neste repositório em [src/ssh-brute-force-parser/](#).

1. O objetivo do projeto

Automatizar exatamente a investigação manual que fizemos no Módulo 01, Aula 7, com `grep / awk`:

"Quais IPs mais tentaram login SSH, quantas vezes, com quais usuários, e algum deles teve sucesso depois de várias falhas?"

2. Desenhando a solução antes de codar

Todo script começa com um plano, não com código. O nosso:

1. Ler o log, linha por linha
2. Para cada linha:
 - Se for uma falha de login → extrair IP e usuário, contar
 - Se for um sucesso de login → guardar IP e usuário
3. No final: para cada IP com falhas $\geq$ threshold, montar um resumo
4. Imprimir o resumo (e opcionalmente salvar em JSON)

Isso é chamado de **pseudocódigo** — descrever a lógica em português antes de traduzir para Python. É um hábito valioso mesmo depois de programar bem: pensar na lógica separado da sintaxe evita muitos erros.

3. As regex do parser

```
FAILED_RE = re.compile(
    r"Failed password for (?:(?:invalid user )?(?P<usuario>\S+) from "
    r"(?P<ip>\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}) port (?P<porta>\d+)"
)
```

Alguns detalhes novos em relação à Aula 2:

- `(?:invalid user )?`: um grupo **não-capturado** (`(?:)`) e **opcional** (`?`) — casa com "invalid user " se existir (login com usuário que não existe no sistema), mas não quebra se não existir.
- `(?P<usuario>\S+)`: um grupo **nomeado** — em vez de acessar por posição (`match.group(1)`), acessamos por nome (`match.group("usuario")`), o que deixa o código muito mais legível.
- `\S+`: um ou mais caracteres que **não** são espaço (`\S` é o oposto de `\s`).

4. `Counter` — contando ocorrências sem reinventar a roda

```
from collections import Counter

falhas_por_ip = Counter()
falhas_por_ip["203.0.113.45"] += 1
falhas_por_ip["203.0.113.45"] += 1
falhas_por_ip["198.51.100.10"] += 1

print(falhas_por_ip.most_common())
# [('203.0.113.45', 2), ('198.51.100.10', 1)]
```

`Counter` é um `dict` especializado em contar — `.most_common()` já devolve ordenado do maior para o menor, exatamente como fizemos "na mão" com `sort | uniq -c | sort -rn` no Módulo 01, Aula 7. Isso é uma lição importante: **Python tem bibliotecas prontas para tarefas comuns** — antes de escrever sua própria lógica de contagem, vale verificar se já existe algo assim na biblioteca padrão.

5. Rodando o projeto

```
cd src/ssh-brute-force-parser
python3 parser.py exemplo_auth.log --threshold 3
```

Saída esperada (usando o log de exemplo incluído no projeto):

```
IPs com falha de login: 2
Logins bem-sucedidos no log: 2

IPs suspeitos (>= 3 falhas):
  203.0.113.45: 6 falhas, usuários tentados: admin, oracle, postgres, root, test ⚠️ TEVE SUCESSO DEPOIS D
```

Repare no alerta `⚠️ TEVE SUCESSO DEPOIS DAS FALHAS` — esse é o dado mais crítico do relatório: não é só "alguém tentou várias senhas", é "alguém tentou várias senhas **e conseguiu entrar**". Na prática (Módulo 15, Incident Response), esse é o tipo de achado que eleva a severidade de "tentativa" para "possível comprometimento real".

6. Testando com seu próprio log

Se você tiver acesso a um `/var/log/auth.log` real (Módulo 01, Aula 5):

```
sudo python3 parser.py /var/log/auth.log --threshold 5 --json /tmp/relatorio.json
```

(`sudo` porque, dependendo da distribuição, esse arquivo só é legível por root ou pelo grupo `adm` — lembra do princípio do menor privilégio, Módulo 00, Aula 2?)

7. Como estender o projeto (ideias para você praticar)

Não vou implementar essas extensões por você — é um ótimo exercício:

1. Adicionar suporte a filtrar por **janela de tempo** (ex.: só contar falhas nos últimos 10 minutos) — vai precisar aprender a extrair e comparar o timestamp de cada linha.
2. Detectar **múltiplos usuários diferentes testados pelo mesmo IP em pouco tempo** como um sinal extra de suspeita (já capturamos `usuarios_tentados`, falta usar isso numa regra adicional).
3. Cruzar os IPs suspeitos com uma lista de IPs privados (lembra da função `eh_ip_privado` da Aula 1?) — tentativas vindas de dentro da própria rede merecem tratamento diferente de tentativas vindas da internet.

8. Recapitulando

- Um parser real combina: leitura de arquivo, regex com grupos nomeados, contagem (`Counter`), lógica condicional de threshold, e saída estruturada (JSON).
- Planejar em pseudocódigo antes de codar evita erros de lógica.
- "Teve sucesso depois das falhas" é um sinal muito mais crítico do que só contar falhas — é o tipo de correlação que diferencia um alerta de um incidente confirmado.

Próxima aula (fecha o Módulo 06)

Mais dois projetos práticos: um **IOC extractor** (extrai IPs, domínios, hashes e e-mails de qualquer texto) e um **hash checker** (calcula e verifica hashes de arquivos) — fechando o módulo com ferramentas que você vai reutilizar em investigações reais.

Fontes recomendadas

- **Python — documentação oficial do módulo `collections`**: referência completa do `Counter` e outras estruturas. <https://docs.python.org/3/library/collections.html>
- **Python — documentação oficial do módulo `argparse`**: referência da biblioteca usada para os argumentos de linha de comando do script (`--threshold` , `--json`). <https://docs.python.org/3/library/argparse.html>